



Unit 5 Quiz (Friday October 24): Note:

Updated Oct. 23 with an added section on code fences and how to handle those

Completion requirements

Due: Friday, October 24, 2025, 10:20 AM

DSC 360 – Unit 5 Review Guide

Note: Quiz date moved from Wednesday to Friday

The quiz covers everything in Unit 5 on Moodle (“LLM-Assisted Data Extraction”), including the slides **LLM-Assisted Data Extraction**, the two assigned videos, conceptual aspects from Lab 5 Mini, and the code example `ticker1.py`.

Terms to Know

- **unstructured text** – information in free-form language, not organized into fields or tables
- **structured data** – information arranged in consistent fields (e.g., JSON or CSV); enables databases and analytics to use it reliably
- **schema** – a formal specification of required keys, types, and rules for data
- **validation** – checking that data meet schema requirements before use; acts as built-in quality control
- **Pydantic** – Python library for defining and validating data models; enforces type and format rules automatically
- **BaseModel** – a Pydantic class serving as the template (blueprint) for valid records
- **field validator** – a function that cleans or enforces rules for one field during validation
- **regex (regular expression)** – compact notation for describing and testing text patterns
- **nullable field** – a field allowed to be `None` when information is missing
- **temperature** – controls randomness in model output; 0 makes results deterministic and reproducible
- **prompt** – the text instruction guiding what information to extract and how to format it
- **schema-guided generation** – requesting output that matches a specific JSON schema instead of free text

- **Exact Match (EM)** – metric comparing predictions with gold data; *Field EM* = fraction of fields exactly correct, *Row EM* = fraction of rows entirely correct; used to gauge accuracy and spot which fields fail
- **unit test** – a small automatic check verifying that a component behaves correctly; schema validation plays this role for data
- **data pipeline** – a step-by-step process that transforms raw input into clean, structured output for later stages

Structured Extraction Pipeline

- **Unstructured text → extraction (JSON target)** Use a concise prompt that says “return only JSON”. Prefer models that support **tools / JSON schema** so you can pass a schema (from `model_json_schema()`).
- **Schema validation (Pydantic)** `BaseModel` enforces keys, types, and rules; **nullable** fields may be `None`; **validators** normalize/clean (e.g., ticker case, price bounds). This is your built-in **unit test** per record.
- **Structured record (JSON object)** Treat the validated object as the source of truth. (You can export to CSV later, but the pipeline conceptually centers on JSON.)
- **Evaluation (Exact Match)** **Field EM** = fraction of fields exactly correct; **Row EM** = fraction of rows fully correct. These scores tell you *where* extraction fails (specific fields vs whole records) and whether your prompt/schema tweaks help.

From the videos / ticker1.py to notice

- **Model choice & determinism:** pick a model that supports **tools/JSON schema**; use **temperature = 0** for reproducibility.
- **Type coercion & cleaning:** e.g., phone numbers or prices can be coerced to numeric; validators handle patterns (regex) and bounds.
- **JSON parsing gotcha:** model output is a *string*—`model_validate_json()` or `json.loads()` turns it into a dict/object before DataFrame use.
- **Single vs multiple rows:** one dict → `pd.DataFrame([dict])` (or set `index=[0]`); lists of dicts work directly.
- **“Required” isn’t reality:** even with required fields, models may emit empty strings—your validators/nullable design decide how to handle this.

Handling code fences in LLM output

The problem. LLMs often wrap JSON in **Markdown code fences** (triple backticks), e.g. `json ...`. These fences are *for display only*—they make text render nicely in chat or documentation but are **not part of valid JSON data**. If you pass a fenced block directly to `json.loads()` or `BaseModel.model_validate_json()`, it will usually raise an error.

Understanding how JSON is represented requires thinking carefully about types. You may encounter “JSON” in several forms:

- **JSON string** – plain text that follows JSON syntax exactly (e.g., `'{"ticker": "AAPL", "price": 225.03}'`).
- **Markdown string** – the same JSON wrapped in code fences (`json ...`); looks valid but must be cleaned first.
- **Python object** – a parsed dictionary or list already loaded into memory (e.g., `{"ticker": "AAPL", "price": 225.03}`).

How to fix it. Before parsing or validation, **strip any Markdown code fences** and then pass the cleaned string to `json.loads()` or `model_validate_json()`.

Rule of thumb: only *plain JSON strings* or *Python objects* can be parsed or validated directly—Markdown-formatted code blocks cannot.

Essay Question (for next Paper Test)

Step Back and Reflect: Why do we need data extraction pipelines?

Modern language models are powerful **narrative engines**—they generate fluent text but are **not inherently reliable**. Left unguided, they may invent structure, drop fields, or misformat data. As engineers, we design **processes that constrain and verify** their output: explicit JSON prompts, deterministic settings, schema validation, and measurable metrics. These elements turn an unpredictable model into a dependable component within a system. The goal isn’t perfection—it’s to produce **predictable, inspectable data** that downstream code can trust. In other words, we raise the probability of usable results instead of “praying to the LLM gods” as ML engineers often joke about.

Example Question

Explain how the **Structured Extraction Pipeline** converts an unreliable text generator into a dependable data-producing system. In your answer, connect (a) the limits of LLMs as narrative models, (b) the roles of schema design, validation, and determinism, and (c) how **Exact Match** metrics guide iteration. Argue why this approach is preferable to ad-hoc prompting—even when the ad-hoc approach “seems to work”—and include one concrete failure mode that the pipeline catches early.